МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ

Университет ИТМО

Дисциплина «Встроенные системы»

ЛАБОРАТОРНАЯ РАБОТА

«Разработка игры Змейка на микроконтроллере STM32 с использованием FreeRTOS»

Вариант: 1

Выполнил(и):
Мантуш Даниил Валерьевич
Пиляров Руслан Назирович
Ануфриев Андрей Сергеевич
Группа: P3319
Проверил:
Быковский Сергей Вячеславович

Санкт-Петербург

2026

1. Цель и задачи работы

Цель работы - разработать и исследовать встроенное приложение «Змейка» для микроконтроллера STM32 с использованием операционной системы реального времени FreeRTOS, обеспечив обработку пользовательского ввода, игровую логику, вывод графики на OLED-дисплей и управление периферийными устройствами.

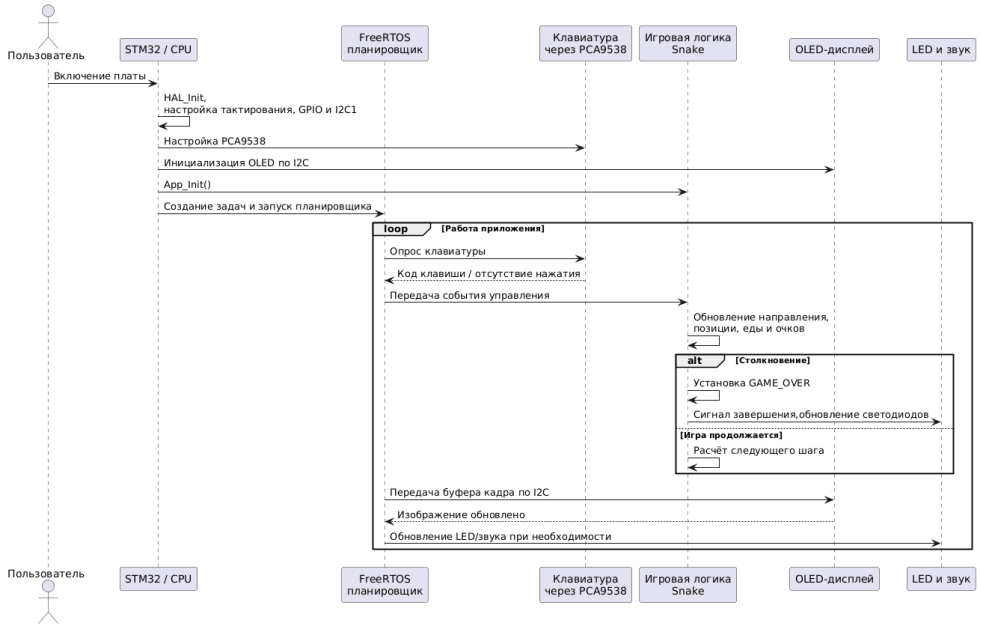
Для достижения цели необходимо решить следующие задачи:

- инициализировать микроконтроллер, GPIO и аппаратный интерфейс I²C1;
- организовать параллельное выполнение программных задач средствами FreeRTOS;
- реализовать чтение матричной клавиатуры через расширитель портов PCA9538;
- разработать игровую логику: движение змейки, появление еды, подсчёт очков, обработку столкновений, паузы и победы;
- реализовать формирование изображения в буфере кадров и передачу его на OLED-дисплей;
- обеспечить звуковую индикацию и отображение состояния игры светодиодами;
- подготовить руководство пользователя и сценарии проверки основных функций приложения.

2. Схема взаимодействия процессора микроконтроллера и периферийных устройств

На этапе запуска микроконтроллер инициализирует HAL, тактирование, GPIO, I²C1 и драйверы платы. После инициализации приложения создаются задачи FreeRTOS, затем запускается планировщик. Во время игры задачи обрабатывают пользовательский ввод, выполняют игровую логику и обновляют дисплей. Доступ к общим данным синхронизируется средствами RTOS.

Диаграмма последовательностей



3. Описание процессов, изображённых на схеме

3.1. Инициализация микроконтроллера и периферии

После сброса выполняется HAL_Init(), затем SystemClock_Config() настраивает системное тактирование от внутреннего генератора HSI. Далее инициализируются GPIO и I²C1. Интерфейс I²C1 работает в режиме 100 кГц. Функция Board_Init() подготавливает линии I²C и расширитель портов PCA9538, используемый для работы с периферией платы.

Драйвер платы настраивает линии клавиатуры и дисплея. Для обмена с расширителями портов и OLED-контроллером используются операции HAL_I2C_Mem_Read() и HAL_I2C_Mem_Write(). При ошибках обмена увеличивается счётчик ошибок I²C; критические ошибки инициализации приводят к вызову Error_Handler().

3.2. Организация задач FreeRTOS

В main() создаются три задачи: defaultTask с приоритетом osPriorityAboveNormal и стеком 384 слова, myTask02 с приоритетом osPriorityNormal и стеком 512 слов, а также displayTask с приоритетом osPriorityBelowNormal и стеком 512 слов. После создания задач запускается планировщик osKernelStart(), который распределяет время процессора между готовыми к выполнению задачами.

Такое разделение позволяет отделить обработку ввода и игрового состояния от вывода изображения. Конкретные периоды задержек и правила обмена между задачами определяются реализацией в app_game.c; события управления и общие данные синхронизируются механизмами FreeRTOS, включая очередь и средства взаимного исключения там, где они используются.

3.3. Опрос клавиатуры

Клавиатура считывается через расширитель портов PCA9538. Программа поочерёдно активирует строки матрицы и считывает входные линии столбцов. По сочетанию строки и столбца определяется код нажатой кнопки. При ошибке чтения драйвер возвращает признак ошибки, чтобы приложение могло обработать некорректный ввод.

Клавиши используются для управления направлением змейки, запуска игры, паузы и повторного старта.

3.4. Игровая логика

Состояние игры хранится в структуре Snake. При инициализации задаются начальная длина змейки, направление движения, начальное состояние GAME_READY и положение еды. Псевдослучайное положение еды выбирается среди свободных клеток игрового поля.

На каждом игровом шаге вычисляется новая позиция головы. Если она выходит за границы поля или пересекается с телом змейки, игра переходит в состояние GAME_OVER. При попадании на еду длина и счёт увеличиваются, обновляется рекорд, после чего размещается новая еда. Если заполнено всё поле, устанавливается состояние GAME_WON.

Скорость движения зависит от набранных очков: период уменьшается на 20 мс за каждые три очка, начиная с 300 мс, но не становится меньше 80 мс. Функции Snake_TogglePause() и

Snake_Turn() реализуют переключение паузы и изменение направления с защитой от мгновенного разворота на 180 градусов.

3.5. Отрисовка OLED-дисплея

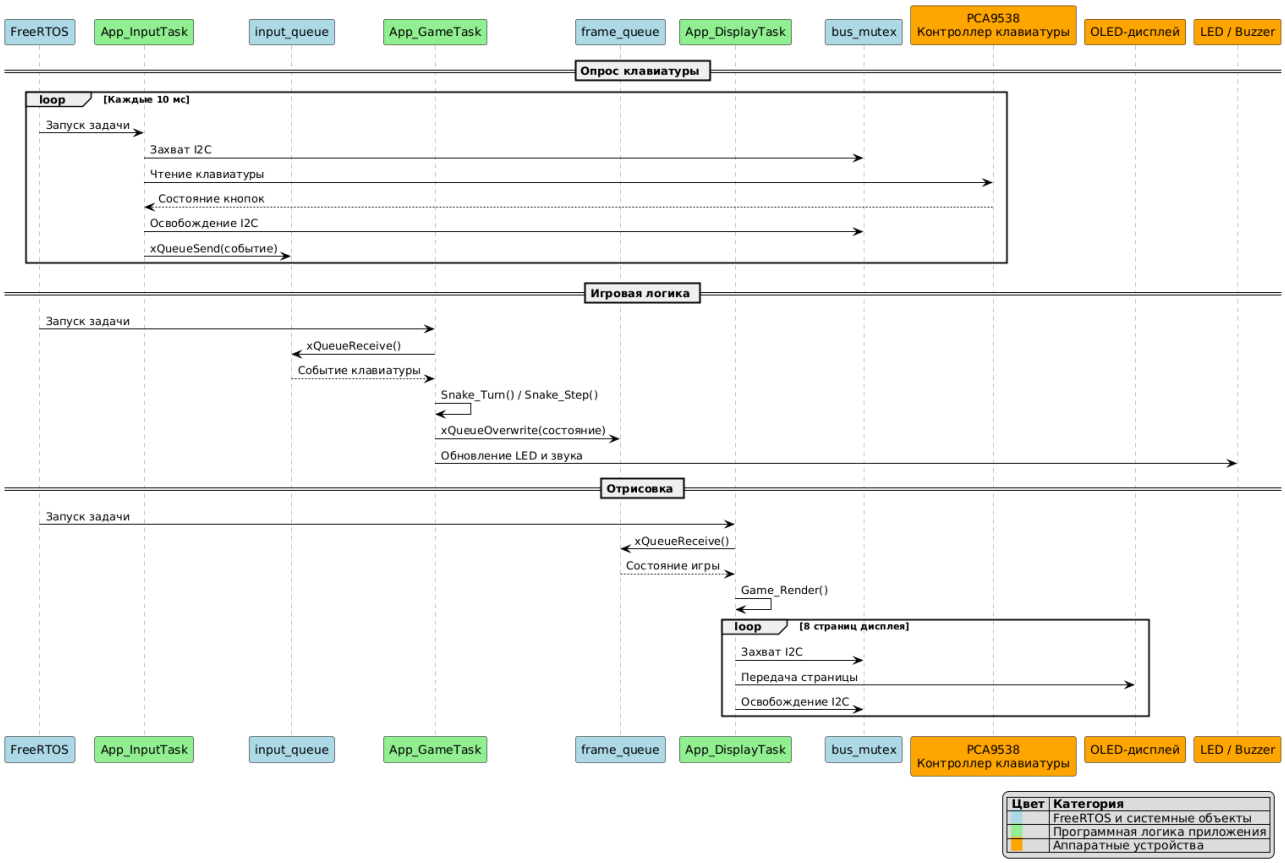
Функция Game_Render() формирует изображение в буфере кадров размером 1024 байта для дисплея 128×64 пикселя. На экране выводятся текущий счёт и рекорд, границы игрового поля, змейка и еда. Для отображения стартового экрана, паузы, победы и проигрыша поверх игрового поля выводится соответствующее сообщение.

Передача изображения выполняется по I²C постранично: для каждой из восьми страниц задаётся адрес, затем передаются 128 байт пиксельных данных. Отдельная задача дисплея позволяет обновлять изображение независимо от логики движения.

3.6. Светодиоды и звук

Светодиоды на выводах PD13, PD14 и PD15 используются для индикации состояния игры и ошибок: зелёный сигнал соответствует запущенной игре, отдельные состояния отображаются при ожидании, паузе или завершении. Звуковой драйвер формирует тональные сигналы, в том числе последовательность звуков при завершении игры. Управление звуком выполняется без длительного блокирования основного игрового цикла.

4. Описание взаимодействия задач FreeRTOS в игре «Змейка»



4.1. Опрос клавиатуры и передача событий

За обработку пользовательского ввода отвечает задача App_InputTask. Она периодически выполняется с интервалом 10 мс и считывает состояние клавиатуры, подключённой через расширитель портов PCA9538.

Перед обращением к клавиатуре задача захватывает мьютекс bus_mutex. Это необходимо для исключения одновременного использования шины I²C несколькими задачами. После получения доступа выполняется чтение состояния кнопок через драйвер платы. PCA9538 возвращает информацию о состоянии входных линий, по которой определяется нажатая клавиша.

После завершения чтения задача освобождает мьютекс, позволяя другим задачам обращаться к устройствам на шине I²C. Если обнаружено подтверждённое нажатие, соответствующее событие помещается в очередь input_queue с помощью функции xQueueSend(). Таким образом, задача ввода передаёт информацию игровой логике, не вызывая её функции непосредственно.

4.2. Обработка игровой логики

Основные игровые операции выполняются задачей App_GameTask. Она получает события пользовательского ввода из очереди input_queue посредством функции xQueueReceive(). Полученное событие интерпретируется в соответствии с назначением кнопки.

Для изменения направления движения используется функция Snake_Turn(), а для выполнения игрового шага - Snake_Step(). В процессе обновления состояния рассчитывается новая позиция змейки, обрабатывается сбор еды, изменяются длина змейки и количество очков. При столкновении с границей поля или собственным телом игра завершается.

После изменения игрового состояния задача обновляет информацию для периферийных устройств. Управление светодиодами позволяет отображать состояние игры, а звуковой драйвер используется для формирования звуковых сигналов, например при сборе еды или завершении игры.

Для передачи актуального состояния в задачу отображения используется очередь frame_queue. Функция xQueueOverwrite() записывает в неё новое состояние игры, заменяя предыдущее, если дисплей ещё не получил его. Поскольку очередь рассчитана на один элемент, она хранит наиболее актуальные данные, не накапливая последовательность устаревших кадров.

4.3. Формирование изображения и работа с OLED-дисплеем

За вывод графики отвечает задача App_DisplayTask. Она получает из очереди frame_queue копию актуального состояния игры с помощью xQueueReceive(). На основе полученных данных формируется изображение, содержащее игровое поле, змейку, еду и информацию о текущем состоянии игры.

Для формирования изображения вызывается функция Game_Render(), которая подготавливает буфер кадров. После этого содержимое буфера передаётся OLED-дисплею через драйвер платы и аппаратный интерфейс I²C.

Передача изображения организована постранично: дисплей имеет разрешение 128 × 64 пикселя, поэтому кадр разделяется на восемь страниц по 128 байт данных. Для передачи каждой страницы задача захватывает bus_mutex, выполняет операцию записи и освобождает мьютекс после завершения обращения к шине. Такой порядок предотвращает одновременный доступ задачи отображения и задачи ввода к общей шине I²C.

После передачи изображения задача ожидает появления нового состояния в frame_queue. Если новое состояние ещё не поступило, ожидание может завершиться по тайм-ауту, после чего задача продолжает выполнение в соответствии с алгоритмом программы.

4.4. Планирование задач и синхронизация

Порядок выполнения задач определяется планировщиком FreeRTOS. Он предоставляет процессорное время готовым к выполнению задачам с учётом назначенных приоритетов. Благодаря этому опрос клавиатуры, обновление игровой логики и вывод графики могут выполняться независимо друг от друга.

Очередь input_queue обеспечивает передачу событий клавиатуры игровой задаче, а frame_queue используется для передачи актуального состояния игры задаче отображения. Мьютекс bus_mutex обеспечивает взаимное исключение при работе с общей шиной I²C.

5. Руководство пользователя

5.1. Запуск

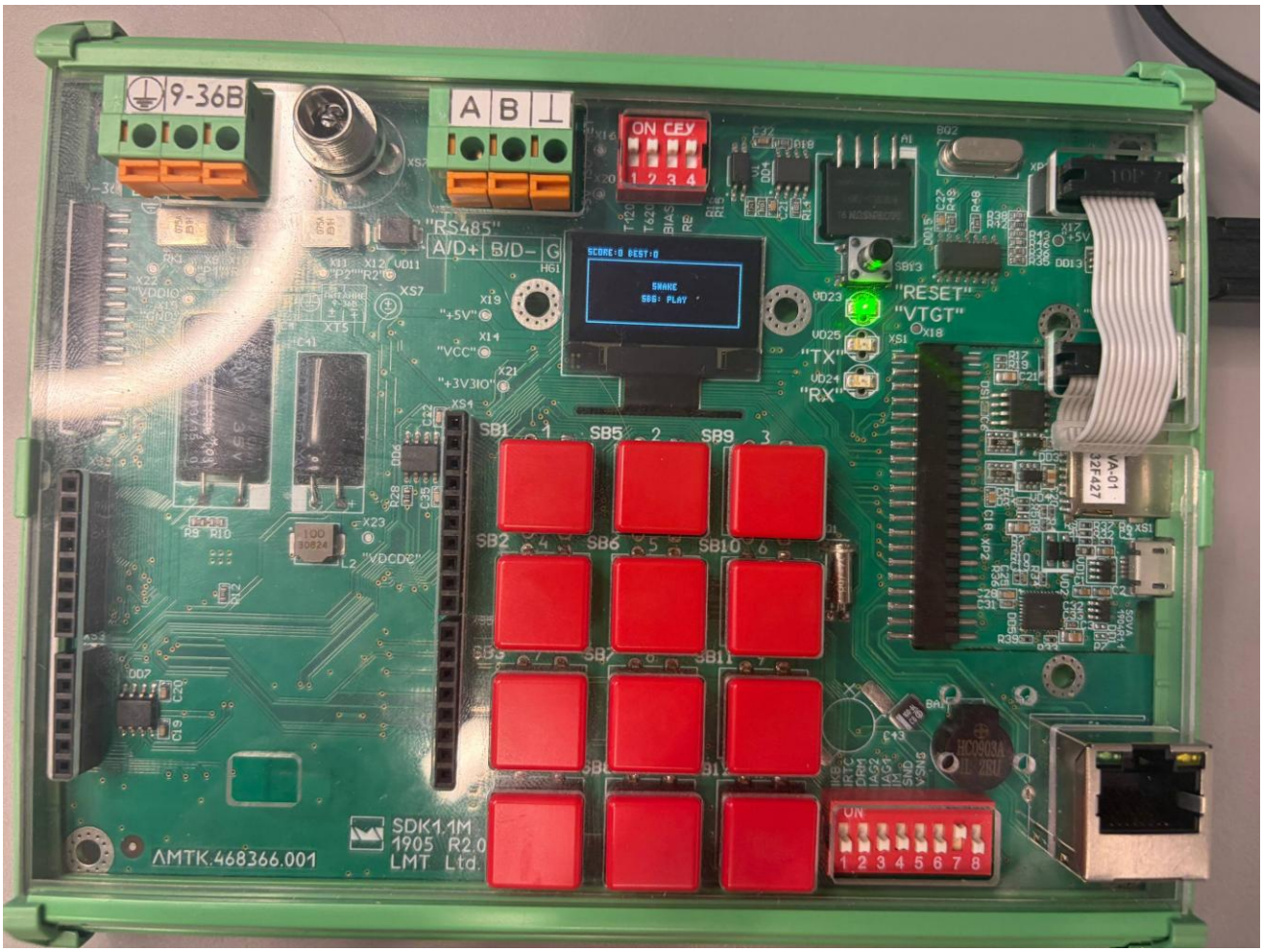
1. Подключить отладочную плату к источнику питания/USB и загрузить собранную прошивку.
2. После инициализации на OLED-дисплее появляется стартовый экран SNAKE.
3. Для начала игры нажать кнопку SB6, как указано в сообщении на дисплее.

5.2. Управление и ход игры

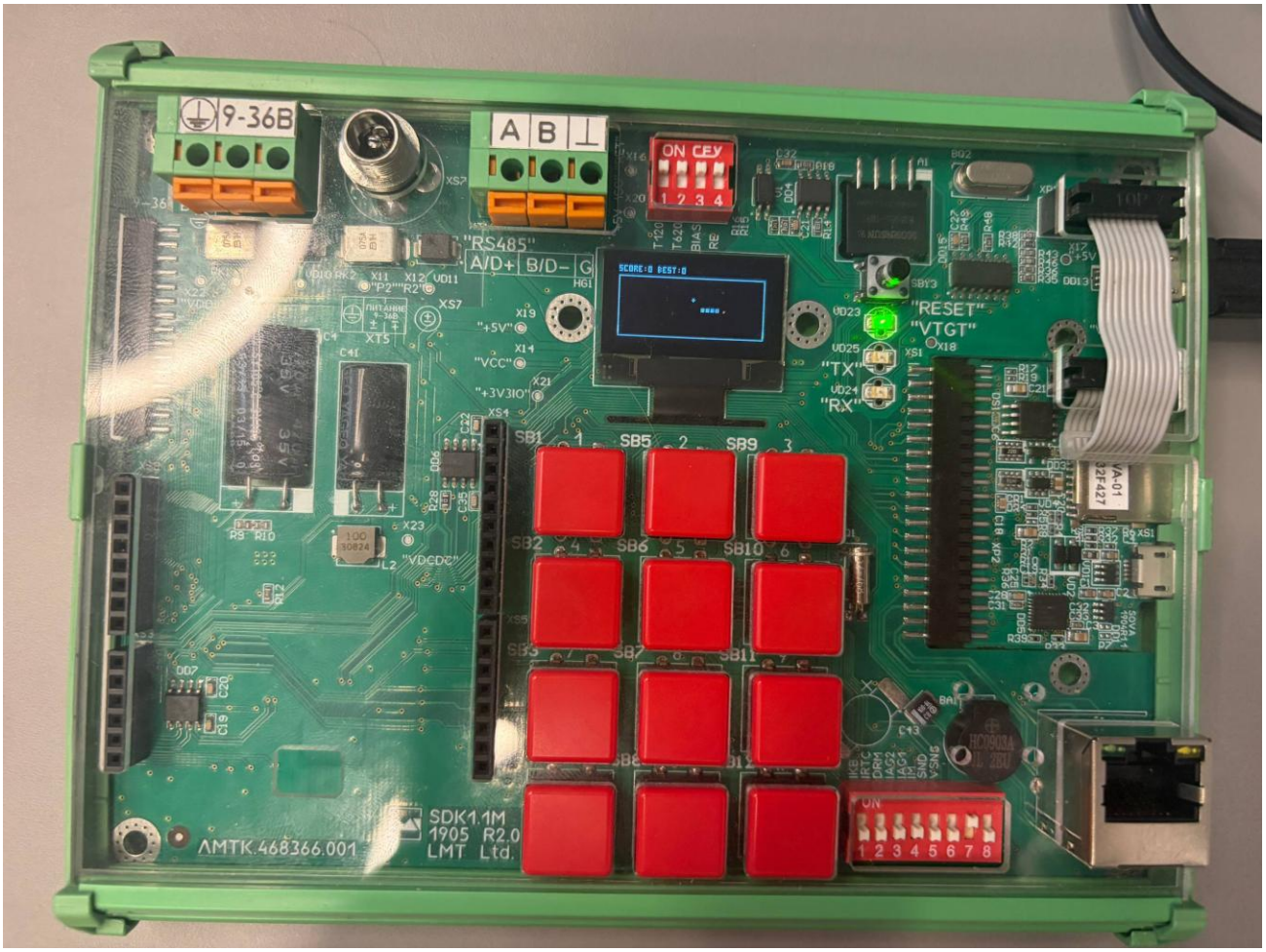
Кнопка	**Действие**
SB1(1)	Не используется
SB5(2)	Движение вверх
SB9(3)	Не используется
SB2(4)	Движение влево
SB6(5)	Пауза/выход из паузы
SB10(6)	Движение вправо
SB3(7)	Не используется
SB7(8)	Движение вниз

SB11(9)	Не используется
SB4(*)	Не используется
SB8(0)	Перезапуск
SB12(#)	Не используется

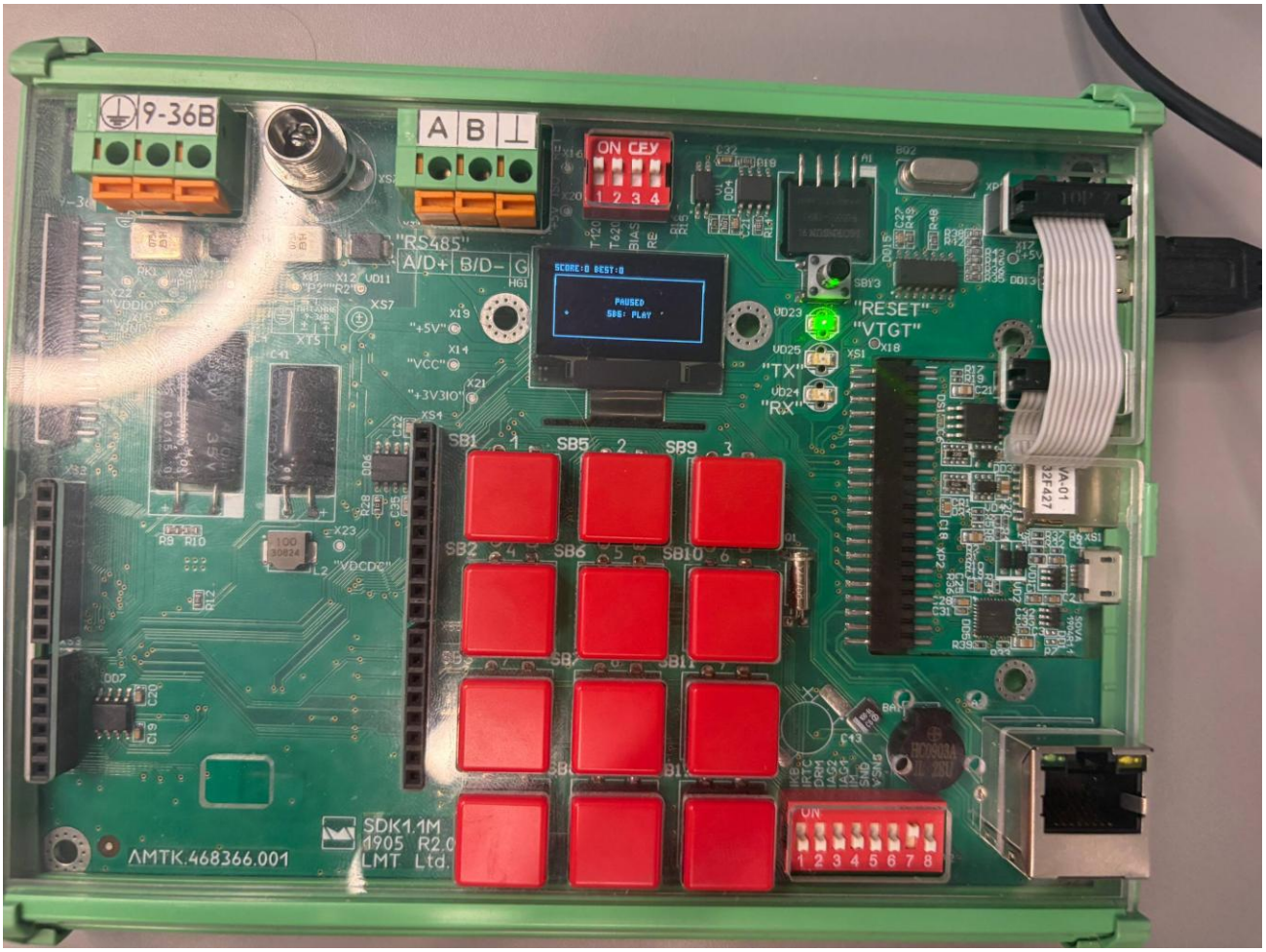
5.3. Демонстрация работы



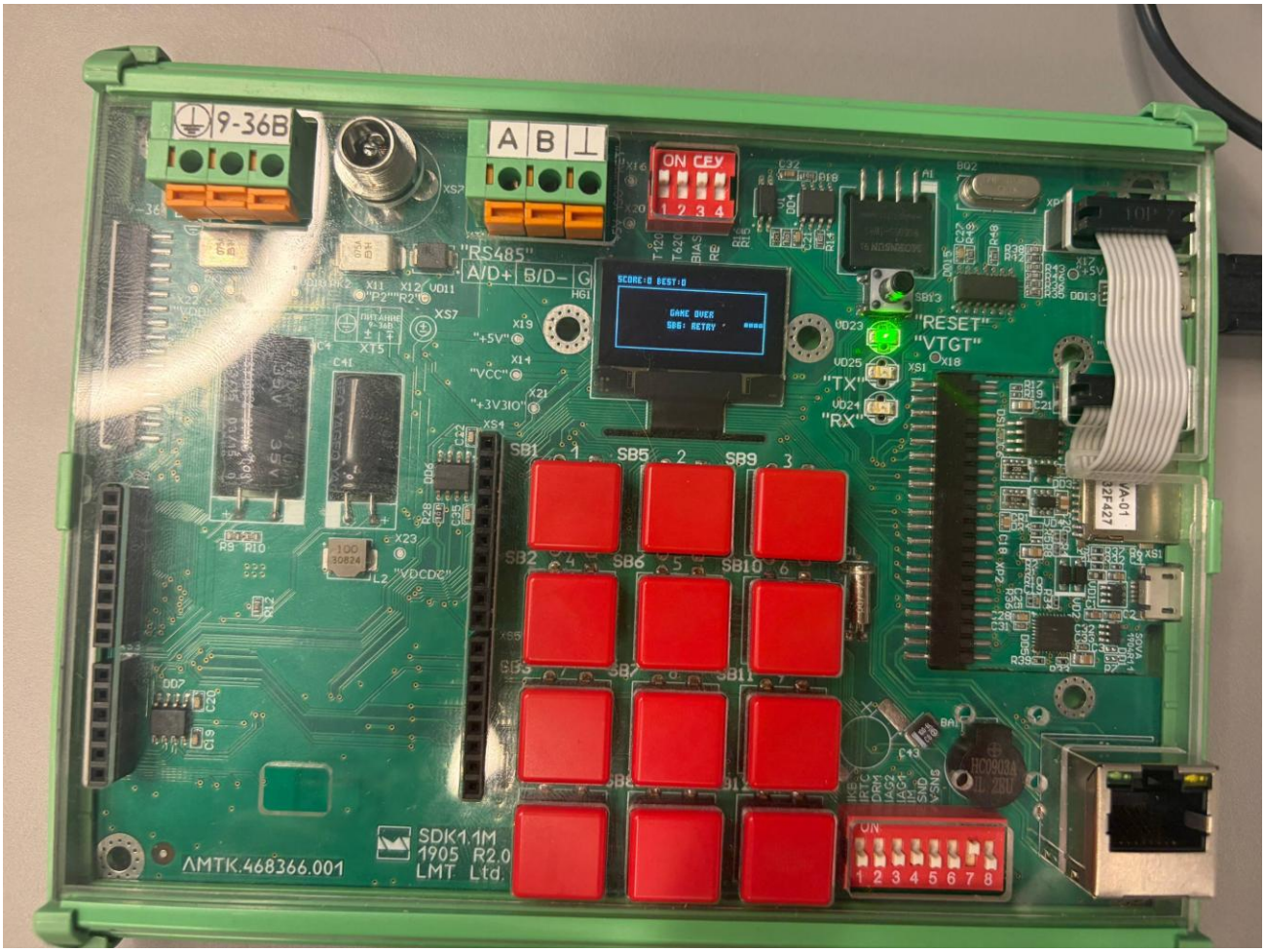
Скриншот 1. Стартовый экран SNAKE перед началом игры.



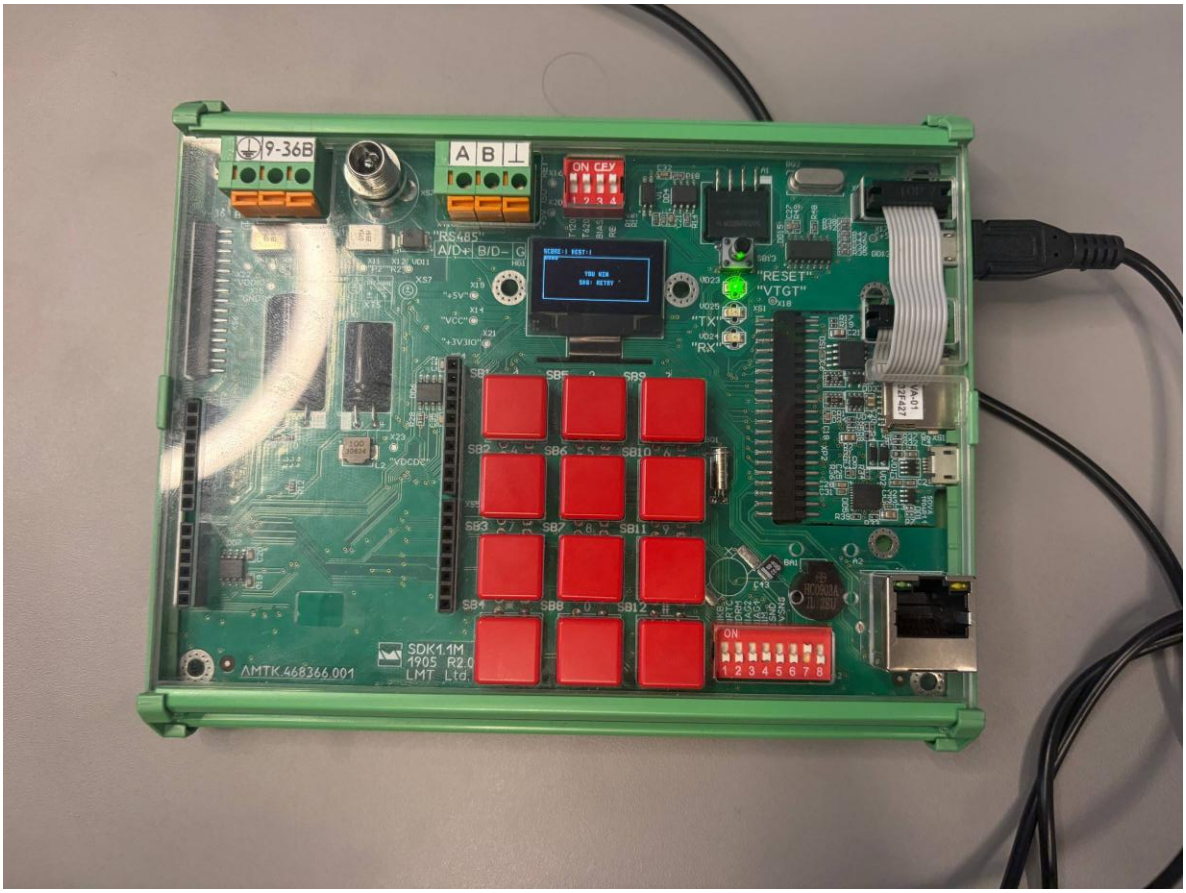
Скриншот 2. Игра в процессе: змейка, еда, текущий счёт и рекорд.



Скриншот 3. Пауза с сообщением PAUSED.



Скриншот 4. Завершение игры: GAME OVER (столкновение).



Скриншот 5. Завершение игры: YOU WIN.

6. Ссылка на репозиторий с кодом

Репозиторий с исходным кодом проекта:

https://drive.google.com/file/d/1rnQQF6fRBgqJBGdzU-3uwqis5nB6UY_b/view?usp=sharing